

Math 502

Homework #3

Name: Gianluca Crescenzo

Exercise 2.7.

- (a) Write a program to solve the boundary value problem for the nonlinear pendulum as discussed in the text. See if you can find yet another solution for the boundary conditions illustrated in Figures 2.4 and 2.5
- (b) Find a numerical solution to this BVP with the same general behavior as seen in Figure 2.5 for the case of a longer time interval, say $T = 20$, again with $\alpha = \beta = 0.7$. Try larger values of T . What does $\max_i \theta_i$ approach as T is increased? Note that for large T this solution exhibits "boundary layers".

Solution. (a) Unsure of the correctness of the following MATLAB code. Below is an attempt to solve the boundary value problem from the text.

```
1  ax = 0;
2  bx = 3;          % T
3  alpha = 1.0;     % alpha = theta(0)
4  beta = 2.0;      % beta = theta(T)
5
6  %grid sizes
7  m1vals = [10 20 40 80 160 1000];
8  ntest = length(m1vals);
9
10 Uprev = [];
11 xprev = [];
12
13 for jtest=1:ntest
14  m1 = m1vals(jtest);
15  m2 = m1 + 1;
16  m = m1 - 1; % number of interior grid points
17
18 % set grid points:
19 gridchoice = 'uniform';
20 x = xgrid(ax,bx,m,gridchoice);
21
22 % 2nd-derivative stencil weights on grid:
23 w2 = zeros(m2,3);
24 for i=2:m1
25  w2(i,:) = fdcoeffF(2, x(i), x((i-1):(i+1)));
26 end
27
```

```

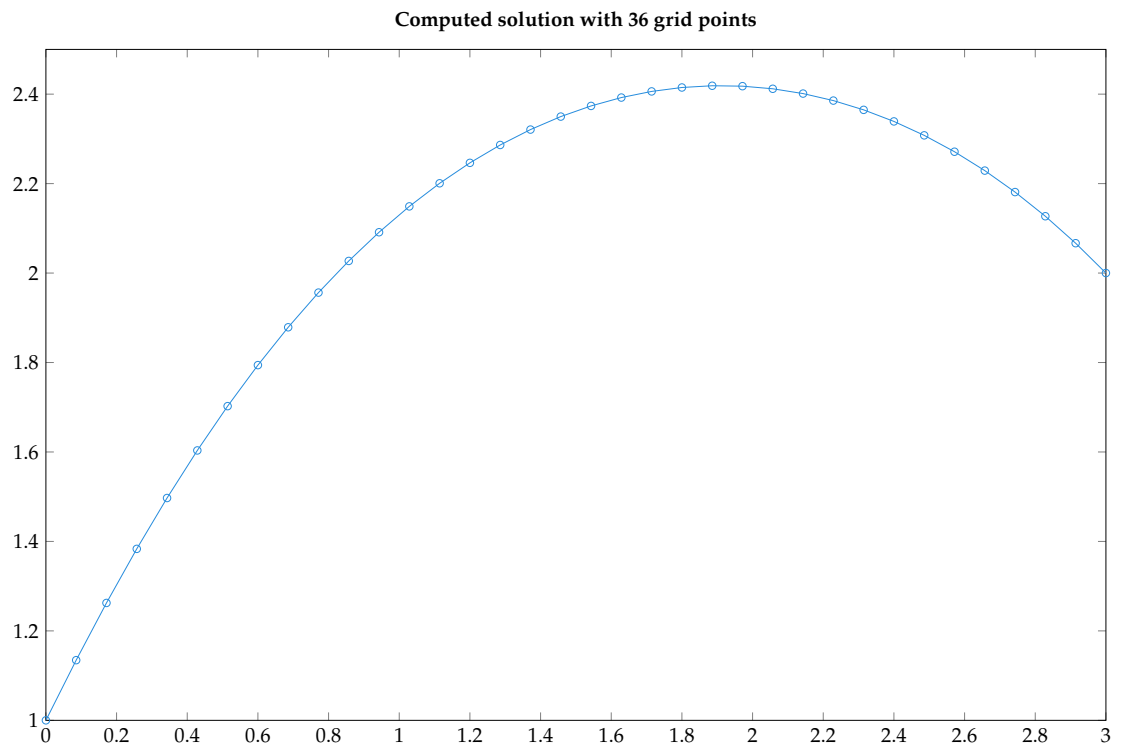
28 % initial guess
29 if ~isempty(Uprev)
30 U = interp1(xprev, Uprev, x, 'linear', 'extrap');
31 U(1) = alpha;
32 U(m2) = beta;
33 else
34 U = alpha + (beta-alpha)*(x-ax)/(bx-ax); % linear interpolation
35 end
36
37 % newton iteration
38 tol = 1e-12;
39 maxit = 30;
40
41 for it=1:maxit
42 % jacobian matrix
43 A = spalloc(m2,m2,3*m2);
44 F = zeros(m2,1);
45
46 % first row w/ dirichlet condition
47 A(1,1) = 1;
48 F(1) = U(1) - alpha;
49
50 % interior rows:
51 for i=2:m1
52     A(i,i-1:i+1) = w2(i,:);
53     A(i,i) = A(i,i) + cos(U(i));
54     F(i) = w2(i,:)*U(i-1:i+1) + sin(U(i));
55 end
56
57 % last row w/ dirichlet condition
58 A(m2,m2) = 1;
59 F(m2) = U(m2) - beta;
60
61 % update
62 dU = -A\F;
63 U = U + dU;
64
65 if norm(dU,inf) < tol
66     break
67 end
68 end
69
70 disp(' ')

```

```

71 disp(sprintf('Computed solution with %i grid points (Newton iters =
    %i)',m2,it))
72
73 clf
74 plot(x,U,'o-')
75 title(sprintf('Computed solution with %i grid points',m2));
76 drawnow
77 input('Hit <return> to continue ');
78
79 % save for next grid as initial guess:
80 Uprev = U;
81 xprev = x;
82 end

```



(b) Unsure of how to solve this.

Exercise 3.1. The MATLAB script `poisson.m` solves the Poisson problem on a square $m \times m$ grid with $\Delta x = \Delta y = h$, using the 5-point Laplacian. It is set up to solve a test problem for which the exact solution is $u(x,y) = \exp(x + y/2)$, using Dirichlet boundary conditions and the right hand side $f(x,y) = 1.25 \exp(x + y/2)$.

- (a) Test this script by performing a grid refinement study to verify that it is second order accurate.
- (b) Modify the script so that it works on a rectangular domain $[a_x, b_x] \times [a_y, b_y]$, but still with $\Delta x = \Delta y = h$. Test your modified script on a non-square domain.

Solution. The following code performs a grid refinement study:

```

1  % poisson2.m  -- solve the Poisson problem u_{xx} + u_{yy} = f(x,y)
2  % on [a,b] x [a,b].
3  %
4  % The 5-point Laplacian is used at interior grid points.
5  % This system of equations is then solved using backslash.
6
7  mvals = [10 20 40 80 160];      % interior grid sizes
8  nm = length(mvals);
9
10 hs = zeros(nm,1);
11 Einf = zeros(nm,1);
12
13 for k=1:nm
14     m=mvals(k);
15
16     a = 0;
17     b = 1;
18     h = (b-a)/(m+1);
19     hs(k) = h;
20     x = linspace(a,b,m+2);      % grid points x including boundaries
21     y = linspace(a,b,m+2);      % grid points y including boundaries
22
23
24     [X,Y] = meshgrid(x,y);       % 2d arrays of x,y values
25     X = X';                      % transpose so that X(i,j),Y(i,j) are
26     Y = Y';                      % coordinates of (i,j) point
27
28     Iint = 2:m+1;                % indices of interior points in x
29     Jint = 2:m+1;                % indices of interior points in y
30     Xint = X(Iint,Jint);         % interior points
31     Yint = Y(Iint,Jint);
32
33     f = @(x,y) 1.25*exp(x+y/2);  % f(x,y) function
34

```

```

35 rhs = f(Xint,Yint);           % evaluate f at interior points for
    right hand side
36                               % rhs is modified below for boundary
                                conditions.
37
38 utrue = exp(X+Y/2);           % true solution for test problem
39
40 % set boundary conditions around edges of usoln array:
41
42 usoln = utrue;                 % use true solution for this test
    problem
43                               % This sets full array, but only
                                boundary values
44                               % are used below. For a problem where
                                utrue
45                               % is not known, would have to set each
                                edge of
46                               % usoln to the desired Dirichlet
                                boundary values.
47
48
49 % adjust the rhs to include boundary terms:
50 rhs(:,1) = rhs(:,1) - usoln(Iint,1)/h^2;
51 rhs(:,m) = rhs(:,m) - usoln(Iint,m+2)/h^2;
52 rhs(1,:) = rhs(1,:) - usoln(1,Jint)/h^2;
53 rhs(m,:) = rhs(m,:) - usoln(m+2,Jint)/h^2;
54
55
56 % convert the 2d grid function rhs into a column vector for rhs of
    system:
57 F = reshape(rhs,m*m,1);
58
59 % form matrix A:
60 I = speye(m);
61 e = ones(m,1);
62 T = spdiags([e -4*e e],[-1 0 1],m,m);
63 S = spdiags([e e],[-1 1],m,m);
64 A = (kron(I,T) + kron(S,I)) / h^2;
65
66
67 % Solve the linear system:
68 uvec = A\F;
69

```

```

70 % reshape vector solution uvec as a grid function and
71 % insert this interior solution into usoln for plotting purposes:
72 % (recall boundary conditions in usoln are already set)
73
74 usoln(Iint,Jint) = reshape(uvec,m,m);
75 err_int = usoln(Iint,Jint) - utrue(Iint,Jint);
76 Einf(k) = max(abs(err_int(:)));
77
78 % assuming true solution is known and stored in utrue:
79 err = max(max(abs(usoln-utrue)));
80 fprintf('Error relative to true solution of PDE = %10.3e \n',err)
81
82 % plot results:
83
84 clf
85 hold on
86
87 % plot grid:
88 % plot(X,Y,'g'); plot(X',Y', 'g')
89
90 % plot solution:
91 contour(X,Y,usoln,30, 'k')
92
93 axis([a b a b])
94 daspect([1 1 1])
95 title('Contour plot of computed solution')
96 hold off
97 end
98 pinf = log(Einf(1:end-1)./Einf(2:end)) ./ log(hs(1:end-1)./hs(2:end
    ));
99 T = table(hs, Einf, 'VariableNames', {'h','Einf'});
100 disp(T)

```

```
>> hw4pt2
Error relative to true solution of PDE = 1.184e-04
Error relative to true solution of PDE = 3.273e-05
Error relative to true solution of PDE = 8.601e-06
Error relative to true solution of PDE = 2.205e-06
Error relative to true solution of PDE = 5.584e-07
```

h	Einf
-----	-----
0.090909	0.00011842
0.047619	3.2732e-05
0.02439	8.6014e-06
0.012346	2.2054e-06
0.0062112	5.5835e-07

```
>>
```

Note that over each iteration, h decreases by a factor of 2 and E_∞ decreases by a factor of 4. Hence $E_\infty \propto h^2$.

(b) Below is the modified script on a non-square domain.

```
1  ax = 0;
2  bx = 1;
3  ay = 0;
4  by = 2;
5
6  mx = 20;           % # interior points in x
7  my = 41;           % # interior points in y
8  h = (bx-ax)/(mx+1); % grid spacing
9
10 x = linspace(ax,bx,mx+2); % grid points x including boundaries
11 y = linspace(ay,by,my+2); % grid points y including boundaries
12
13
14 [X,Y] = meshgrid(x,y); % 2d arrays of x,y values
15 X = X';                % transpose so that X(i,j),Y(i,j) are
16 Y = Y';                % coordinates of (i,j) point
17
18 Iint = 2:mx+1;        % indices of interior points in x
19 Jint = 2:my+1;        % indices of interior points in y
20 Xint = X(Iint,Jint);  % interior points
21 Yint = Y(Iint,Jint);
22
23 f = @(x,y) 1.25*exp(x+y/2); % f(x,y) function
24
```

```

25 rhs = f(Xint,Yint);           % evaluate f at interior points for
    right hand side
26                               % rhs is modified below for boundary
                                conditions.
27
28 utrue = exp(X+Y/2);           % true solution for test problem
29
30 % set boundary conditions around edges of usoln array:
31
32 usoln = utrue;                % use true solution for this test
    problem
33                               % This sets full array, but only
                                boundary values
34                               % are used below. For a problem where
                                utrue
35                               % is not known, would have to set each
                                edge of
36                               % usoln to the desired Dirichlet
                                boundary values.
37
38
39 % adjust the rhs to include boundary terms:
40 rhs(:,1) = rhs(:,1) - usoln(Iint,1)/h^2;
41 rhs(:,my) = rhs(:,my) - usoln(Iint,my+2)/h^2;
42 rhs(1,:) = rhs(1,:) - usoln(1,Jint)/h^2;
43 rhs(mx,:) = rhs(mx,:) - usoln(mx+2,Jint)/h^2;
44
45
46 % convert the 2d grid function rhs into a column vector for rhs of
    system:
47 F = reshape(rhs,mx*my,1);
48
49 % form matrix A:
50 Ix = speye(mx);
51 Iy = speye(my);
52 ex = ones(mx,1);
53 ey = ones(my,1);
54 T = spdiags([ex -4*ex ex],[-1 0 1],mx,mx);
55 S = spdiags([ey ey],[-1 1],my,my);
56 A = (kron(Iy,T) + kron(S,Ix)) / h^2;
57
58
59 % Solve the linear system:

```



```

60 uvec = A\F;
61
62 % reshape vector solution uvec as a grid function and
63 % insert this interior solution into usoln for plotting purposes:
64 % (recall boundary conditions in usoln are already set)
65
66 usoln(Iint,Jint) = reshape(uvec,mx,my);
67
68 % assuming true solution is known and stored in utrue:
69 err = max(max(abs(usoln-utrue)));
70 fprintf('Error relative to true solution of PDE = %10.3e \n',err)
71
72 % plot results:
73
74 clf
75 hold on
76
77 % plot grid:
78 % plot(X,Y,'g'); plot(X',Y','g')
79
80 % plot solution:
81 contour(X,Y,usoln,30,'k')
82
83 axis([ax bx ay by])
84 daspect([1 1 1])
85 title('Contour plot of computed solution')
86 hold off

```

